

[Before you start](#)

[Download project starter files and deploy Azure services](#)

[Complete the YAML deployment files and deploy to AKS](#)

[Run the client app](#)

[Clean up resources](#)

[Troubleshooting](#)

Deploy an AI inference API to Azure Kubernetes Service

In this exercise, you deploy Azure resources including a Microsoft Foundry AI model, Azure Container Registry (ACR), and Azure Kubernetes Service (AKS) cluster. You then complete Kubernetes manifest files to define container specifications, health probes, resource limits, and load balancing. After deploying the containerized API to AKS, you use a Python client application to test the deployed API endpoints including health checks, readiness validation, and AI model inference requests.

Tasks performed in this exercise:

- Download the project starter files
- Deploy resources to Azure
- Complete the *deployment.yaml* and *service.yaml* files and deploy the container to AKS
- Run the client app to test the API

This exercise takes approximately **30** minutes to complete.

Important: Azure Container Registry task runs are temporarily paused from Azure free credits. This exercise requires a Pay-As-You-Go, or another paid plan.

Before you start

To complete the exercise, you need:

- An Azure subscription with the permissions to deploy the necessary Azure services. If you don't already have one, you can [sign up for one](#).
- [Visual Studio Code](#) on one of the [supported platforms](#).
- The latest version of the [Azure CLI](#).
- The Kubernetes command-line tool, [kubectl](#).
- [Python 3.12](#) or greater.

Download project starter files and deploy Azure services

In this section you download the starter files for the console app and use a script to deploy the necessary services to your Azure subscription. The Azure resource deployments can take 15-20 minutes to complete.

1. Open a browser and enter the following URL to download the starter file. The file will be saved in your default download location.

Code	 Copy
<code>https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/aks-deploy-python.zip</code>	

2. Copy, or move, the file to a location in your system where you want to work on the project. Then unzip the file into a folder.
3. Launch Visual Studio Code (VS Code) and select **File > Open Folder...** in the menu, then choose the folder containing the project files.
4. Open the *azdeploy.py* deployment script and change the resource group and location values at the top of the script to meet your needs, then save your changes. **Note:** Do not change anything else in the script.

Code	 Copy
<pre>rg = "<your-resource-group-name>" # Resource Group name location = "<your-azure-region>" # Azure region for the resources</pre>	

Note: It is recommended to use one of the following three Azure regions for deployment: **eastus2**, **swedencentral**, or **australiaeast**. These regions support the deployment of the AI inference model used in the exercise.

5. In the menu bar select **Terminal > New Terminal** to open a terminal window in VS Code.
6. Run the following command to login to your Azure account. Answer the prompts to select your Azure account and subscription for the exercise.

CodeCopy

```
az login
```

7. Run the following commands to ensure your subscription has the necessary resource providers to install AKS, ACR, and the Foundry AI model. The **Microsoft.Compute**, **Microsoft.Network**, and **Microsoft.Storage** providers are dependencies of AKS that Azure usually registers automatically, but explicitly registering them avoids occasional cluster creation failures on new subscriptions.

CodeCopy

```
az provider register --namespace Microsoft.CognitiveServices
az provider register --namespace Microsoft.ContainerService
az provider register --namespace Microsoft.ContainerRegistry
az provider register --namespace Microsoft.Compute
az provider register --namespace Microsoft.Network
az provider register --namespace Microsoft.Storage
```

8. Make sure you are in the root directory of the project and run the following command to launch the deployment script.

CodeCopy

```
python azdeploy.py
```

Note: Depending on your system configuration, the Python command might be **python3** instead of **python**.

Deploy resources to Azure

With the deployment script running, follow these steps to create the needed resources in Azure.

1. Enter **1** to launch the **1. Provision gpt-5-mini model in Microsoft Foundry** option. This option creates the resource group if it doesn't already exist, creates the resource in Mlcrosoft Foundry, and deploys the **gpt-5-mini** model to the resource.

Important: If there are errors during the model deployment, enter **7** to launch the **7. Delete/Purge Foundry deployment** option. This will delete the deployment and purge the resource name. Exit the menu, and change the region in the deployment script to one of the other recommended regions. Then restart the deployment script and run the model provisioning option again.

2. After the model is deployed, enter **2** to launch **2. Create Azure Container Registry (ACR)**. This creates the resource where the API container will be stored, and later pulled into the AKS resource.
3. After the ACR resource has been created, enter **3** to launch **3. Build and push API image to ACR**. This option uses ACR tasks to build the image and add it to the ACR repository. This operation can take 3-5 minutes to complete.
4. After the image has been built and pushed to ACR, enter **4** to launch the **4. Create AKS cluster** option. This creates the AKS resource configured with a managed identity and gives the service permission to pull images from the ACR resource. This operation can take 5-10 minutes to complete.
5. After the AKS resources have been deployed, enter **5** to launch the **5. Check deployment status** option. This option reports whether all three resources were successfully deployed.

If all of the services return a **successful** message, enter **9** to exit the deployment script.

If the AKS cluster is in a **Failed** or **Canceled** state, correct the reported issue, then enter **8** to launch the **8. Delete failed AKS deployment** option before running option **4** again. This guarded option does not delete a healthy or in-progress cluster.

Next, you complete the YAML files necessary to deploy the API to AKS.

Complete the YAML deployment files and deploy to AKS

In this section you complete both the *deployment.yaml* and *service.yaml* files. The deployment manifest defines how the API container is deployed and managed in AKS, while the service manifest exposes the API to external traffic through a load balancer.

1. Open the *k8s/deployment.yaml* file to begin completing the file.

Tip: To maintain proper code indentation, paste the code flush with the left margin (column 1), select all of the pasted lines, and press **Tab** to align the block with the **BEGIN / END** markers. Press **Shift+Tab** to outdent if needed.

1. Locate the **# BEGIN: Container specification** comment and add the following YAML section to the manifest under the comment. Ensure YAML indentation is correct.

CodeCopy

```
containers:  # List of containers to run in the pod
- name: api
  image: ACR_ENDPOINT/aks-api:latest  # Container image from ACR
  imagePullPolicy: Always  # Always pull the latest image from registry
  ports:  # Ports exposed by the container
  - name: http
    containerPort: 8000
    protocol: TCP
```

This section defines the container specification, including which container image to use from ACR, the pull policy, and which port the container exposes for HTTP traffic.

2. Locate the **# BEGIN: Liveness Probe Configuration** comment and add the following YAML section to the manifest under the comment. Ensure YAML indentation is correct.

CodeCopy

```
livenessProbe:  # Detects if container is alive or needs restart
  httpGet:
    path: /healthz  # Health check endpoint path
    port: http
  initialDelaySeconds: 10  # Seconds to wait before first check
  periodSeconds: 30
  timeoutSeconds: 5
  failureThreshold: 3  # Consecutive failures before restarting container
```

This section configures the liveness probe, which periodically checks if the container is healthy by making HTTP requests to the **/healthz** endpoint. If the probe fails three consecutive times, Kubernetes automatically restarts the container.

3. Locate the **# BEGIN: Resource Limits Configuration** comment and add the following YAML section to the manifest under the comment. Ensure YAML indentation is correct.

CodeCopy

```
resources:  # CPU and memory resource specifications
  requests:  # Minimum resources guaranteed to the container
    memory: "256Mi"
    cpu: "250m"
  limits:  # Maximum resources the container can use
    memory: "512Mi"
    cpu: "500m"
```

This section defines the CPU and memory resources for the container. Requests specify the minimum resources guaranteed, while limits set the maximum resources the container can consume. This helps Kubernetes schedule pods efficiently and prevents resource starvation.

4. Save your changes and take a few minutes to review the completed *deployment.yaml* file.

Next, you update the *service.yaml* file.

1. Open the *k8s/service.yaml* to complete the file.

2. Add the following YAML to the manifest. Ensure YAML indentation is correct.

Code	Copy
<pre>apiVersion: v1 kind: Service # Service: exposes pods on a network and provides load balancing metadata: name: aks-api-service # Unique name for the service labels: app: aks-api # Matches deployment and pod labels annotations: service.beta.kubernetes.io/azure-load-balancer-internal: "false" # Use public load balancer spec: # Service specification type: LoadBalancer # Exposes service externally selector: # Selects which pods to route traffic to based on labels app: aks-api version: v1 ports: # Port mappings between service and pods - name: http port: 80 # Service port exposed externally targetPort: http # Pod container port to forward traffic to protocol: TCP sessionAffinity: None # Client requests not pinned to specific pods</pre>	

This manifest creates a LoadBalancer Service that exposes your API pods externally through an Azure Load Balancer. It routes incoming traffic on port 80 to the container's port 8000, using label selectors to identify which pods should receive traffic.

3. Save your changes and take a few minutes to review the file.

Apply the manifests to AKS

In this section you use the deployment script to apply the manifests to AKS.

1. Make sure you are in the root directory of the project and run the following command to launch the deployment script.

Code	Copy
<pre>python azdeploy.py</pre>	

2. Enter **6** to launch the **6. Deploy to AKS** option. This option performs several operations: it retrieves your AKS credentials and configures kubectl, assigns the **Cognitive Services OpenAI User** role to the AKS kubelet managed identity so the API can authenticate to Foundry using Microsoft Entra ID, updates the deployment manifest with your ACR endpoint and Foundry endpoint, and then uses **kubectl apply** to deploy both manifests to your AKS cluster. When the operation is complete, enter **9** to exit the deployment script.

3. Run the following commands in the terminal to verify the deployment. Expect **kubectl get deploy,svc** to show the Deployment **READY** as **1/1** (or your replica count) and the Service **EXTERNAL-IP** to have a public IP (not **<pending>**). The rollout command should print **deployment "aks-api" successfully rolled out** when the update is complete.

Code	Copy
<pre>kubectl get deploy,svc kubectl rollout status deploy/aks-api</pre>	

Run the client app

In this section, you configure the Python environment and then perform operations on the API using the client app

Configure the Python environment

In this section, you create the Python environment and install the dependencies.

1. Ensure you are in the *client* folder of the project in the terminal.

2. Run the following command in the VS Code terminal to create the Python environment.

Code	Copy
<pre>python -m venv .venv</pre>	

3. Run the following command to activate the Python environment. **Note:** On Linux/macOS, use the Bash command. On Windows, use the PowerShell command. If using Git Bash on Windows, use **source .venv/Scripts/activate**.

Bash

Code	Copy
<pre>source .venv/bin/activate</pre>	

PowerShell

Code	Copy
<pre>.\.venv\Scripts\Activate.ps1</pre>	

4. Run the following command in the VS Code terminal to install the dependencies.

Code	Copy
<pre>pip install -r requirements.txt</pre>	

Perform operations with the app

Now it's time to run the client application to perform various operations on the API. The app provides a menu-driven interface.

1. Run the following command in the terminal to start the console app. Refer to the commands from earlier in the exercise to activate the environment, if needed, before running the command.

Code	Copy
<pre>python main.py</pre>	

- 2. Enter **1** to start the **1. Check API Health (Liveness)** option. This verifies that the API container is running and responds to health checks, which is the same endpoint used by the Kubernetes liveness probe.
- 3. Enter **2** to start the **2. Check API Readiness (Foundry Connectivity)** option. This confirms the API can successfully connect to the Foundry model endpoint and is ready to process inference requests.
- 4. Enter **3** to start the **3. Send Inference Request** option. This sends a single prompt to the API and receives a complete response from the deployed model. Single inference requests are useful for batch processing, automated tasks, or when you need the entire response at once for further processing.
- 5. Enter **4** to start the **4. Start Chat Session (Streaming)** option. This starts an interactive chat session where responses from the model are streamed in real-time as they're generated.

When you're finished enter **5** to exit the app.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. Run the following command in the VS Code terminal to delete the resource group, and all resources in the group. Replace **<rg-name>** with the name you choose earlier in the exercise. The command will launch a background task in Azure to delete the resource group.

Code	Copy
<pre>az group delete --name <rg-name> --no-wait --yes</pre>	

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.

Troubleshooting

If you encounter issues while completing this exercise, try the following troubleshooting steps:

Verify Azure resource deployment

- Navigate to the [Azure portal](#) and locate your resource group.
- Confirm that the Microsoft Foundry resource shows a **Provisioning State** of **Succeeded** and the **gpt-5-mini** model is deployed.
- Verify the Azure Container Registry (ACR) exists and contains the **aks-api** image.
- Check that the AKS cluster is in a **Succeeded** state and the nodes are running.

Resolve AKS cluster creation failures

- Run the deployment script and select option **5. Check deployment status**. Quota validation can fail before an AKS resource is created, while failures later in provisioning can leave the cluster in a **Failed** or **Canceled** state.
- If the error reports that **Standard_D2s_v7** is unavailable or that the region has insufficient capacity, exit with option **9**. Change **AKS_VM_SIZE** to one of the v5 or v6 fallback sizes listed near the top of the deployment script, or change the **location** value to another recommended region, and then run option **4. Create AKS cluster** again.
- If the error reports insufficient quota, select a region where your subscription has available Dsv5-family quota or request a quota increase. Changing regions only helps when the other region has sufficient quota.
- The AKS cluster uses the **location** configured in the script even when the resource group already exists in another region.
- If option **5** reports **Failed** or **Canceled**, correct the underlying issue and run option **8. Delete failed AKS deployment** before retrying option **4**. If no AKS resource was created, no deletion is needed.

Verify AKS deployment status

- Run **kubectl get pods** to check if the API pods are running. Look for **Running** status.
- Run **kubectl get svc** to verify the LoadBalancer service has an external IP assigned (not **<pending>**).
- Run **kubectl describe pod <pod-name>** to see detailed pod status and events if issues occur.
- Check pod logs with **kubectl logs <pod-name>** to see container startup errors or runtime issues.

Verify YAML file completeness

- Ensure all YAML sections were added correctly to *deployment.yaml* and *service.yaml* between the appropriate comment markers.
- Verify YAML indentation is correct (use spaces, not tabs) as incorrect indentation will cause deployment failures.
- Confirm the ACR endpoint was properly substituted in the deployment manifest by the deployment script.

Verify client connectivity

- Ensure the API endpoint uses the correct external IP from the LoadBalancer service.
- Verify you can reach the API endpoint by running **curl http://<external-ip>/healthz** from the terminal.

Check Python environment and dependencies

- Confirm the virtual environment is activated before running the client app.
- Verify that all packages from *requirements.txt* were installed successfully by running **pip list**.
- Ensure you're running the client from the *client* directory.